DotSlash 9.0
Problem Statements
Hackathon Participation Rules

The following rules apply to all shortlisted teams participating in DotSlash 9.0. These rules govern participant behaviour, development during the hackathon, and general conduct throughout the event. All participants are expected to follow these instructions at all times.

Presence and Participation

All team members must be present during the hackathon as per the schedule shared by the organizers. Teams must participate only with their registered members. Replacement of team members is not allowed unless approved by the organizing committee.

ID Card Requirement

Participants must carry their hackathon ID at all times. ID cards are mandatory for entry, verification, and for receiving food. Participants without valid ID may not be allowed to access food or restricted areas.

Campus Restriction During Development Phase

Participants are not allowed to leave the SVNIT Guest House campus during the 24-hour development phase of the hackathon. Teams must remain within the designated hackathon area unless permitted by the organizers.

Code of Conduct

Participants must maintain professional and respectful behaviour at all times. Any form of harassment, misbehaviour, cheating, or violation of rules may lead to immediate disqualification. Participants must follow instructions given by organizers, mentors, and judges.

Work During Hackathon

All core development must take place during the hackathon duration. Participants may use open-source libraries, APIs, and tools, but the project

should not be pre-built. Minor setup or environment configuration done before the event is acceptable, but the main functionality must be developed during the hackathon.

Use of Internet

Internet access is provided for development, documentation, and API usage. Participants must use the internet responsibly and only for intended hackathon purposes. Any misuse of network access, downloading unrelated content, or activities not related to the hackathon may lead to disciplinary action.

Use of External Resources

Participants are allowed to use open-source libraries, public APIs, AI tools, and online documentation. Copying complete projects, reusing previous hackathon submissions, or using code not written by the team is not allowed.

Team Conduct and Collaboration

Teams must work independently. Collaboration between different teams is not allowed. Sharing code, assets, or implementation with other teams is strictly prohibited. Participants must keep their workstations clean and organized at all times.

Demo and Submission

All teams must submit their project before the announced deadline using the official submission platform. Teams must be ready to demonstrate their project when called by the judges. The project should be functional during evaluation.

Organizer Authority

The organizing committee reserves the right to modify rules, resolve disputes, or take disciplinary action if required. All decisions taken by the organizers and judges will be final.

Most Important Rule

Respect others, follow the rules, build something meaningful, and most importantly, have fun!

General Rules for Problem Statements

Participants are free to choose any problem statement within the provided tracks. The following rules apply to all submissions across the tracks.

1. Originality Requirement

All projects must be **original work** created during the hackathon.
Pre-built projects, previously submitted work, or direct copies of existing applications are not allowed. Also means **no white labelling.**

2. Real-World Relevance

Projects should aim to solve **real and meaningful problems.**
Solutions that demonstrate practical usefulness, scalability, or real-world applicability will be given preference over purely conceptual or demo-only projects.

3. Avoid Overused / Cliché Ideas

Participants are strongly discouraged from building overdone projects such as:

- Basic to-do list or CRUD apps
- Simple chat applications
- Portfolio or static websites
- Generic AI chatbots without innovation
- Simple dashboards with no real functionality
- Basic clones of popular apps
- Token generators or NFT-only projects
- Weather apps, calculator apps, or similar trivial ideas

Projects should demonstrate creativity, technical depth, and originality.

4. Avoid AI Wrappers Without Real Logic

Using AI APIs alone is not sufficient.
Projects that only wrap an AI model without meaningful logic, engineering, or innovation may receive low scores. **Higher value will be given to projects having custom trained ML models.**

AI should be used as a component of the system, not the entire project.

5. Encourage Offline-First and Low-Connectivity Solutions

Participants are encouraged to design systems that can work in:

- Low internet connectivity environments
- Rural or remote areas
- Disaster or emergency situations
- High-load public systems

Offline-first, local-first, or resilient architectures will be considered a positive factor during evaluation.

6. Technical Depth Matters

Projects should demonstrate good software engineering practices such as:

- Proper architecture
- Meaningful backend logic
- Data handling
- Security considerations
- Performance optimization
- Scalability

Projects that only focus on UI without real functionality may be penalized. Also **ensure that the README.md file exists with all the steps to run the project, and a deep explanation of the project itself.**

7. Use of Open Source and APIs

Participants may use open-source libraries, APIs, and frameworks.
However, the **core logic and implementation must be developed by the team.**

Simply combining multiple APIs without meaningful integration will not be considered sufficient.

8. Judging Preference

Judges will give higher preference to projects that show:

- Innovation
- Real-world usefulness
- Technical complexity
- Good design and architecture
- Reliability and stability
- Ability to work in real conditions

9. Track Flexibility

Participants may choose any track that best fits their project, on **first come first serve basis**.

10. Final Decision

The decision of the judges and organizing committee will be **final in all cases.**

DevTech Track

Description

The DevTech track focuses on building tools, platforms, and infrastructure that help developers build, debug, deploy, and manage software efficiently. Projects should target developers, engineering teams, or organizations rather than general users. Strong emphasis should be placed on system design, performance, reliability, and developer experience. Self-hosted, local-first, and offline-capable tools are encouraged.

Example Project Ideas

- Large codebase architecture visualization tool
- Local-first sync engine for collaborative applications
- Self-hosted API gateway with request analytics
- CLI automation framework for developer workflows
- Distributed log search and debugging platform
- Git repository intelligence and insights tool
- Offline documentation search engine
- AI agentic frameworks, context layers, or workflows
- Multi-repo task runner for monorepos
- Dependency vulnerability and license scanner
- Lightweight local CI runner

Suggested Tech Stack

- Go / Rust / Node.js / Python
- PostgreSQL / SQLite / Redis
- gRPC / WebSockets / REST
- Docker APIs / container runtimes
- AST parsers / static analysis libraries
- Git APIs / filesystem watchers
- Next.js / React (dashboard only)
- Local-first storage / sync logic

Projects to Avoid

- Simple code editors
- REST API generators
- Markdown or notes apps
- Basic terminal clones
- Generic AI coding assistants
- GitHub clone projects
- Simple dashboards with fake data

- API wrappers without logic
- Tools without real developer use
- Frontend-only dev tools

GreenTech Track

Description

The GreenTech track focuses on software solutions that improve sustainability, environmental awareness, and efficient resource usage. Projects should address real problems related to climate, energy, waste, water, agriculture, or environmental data analysis using software-only approaches. Solutions that work in low-connectivity, rural, or large-scale environments are encouraged.

Example Project Ideas

- Carbon emission tracking for organizations using real datasets
- Food supply chain waste analysis platform
- Energy usage optimization for buildings using usage logs
- Climate data visualization and risk prediction tool
- Public pollution data aggregation platform
- Smart route planner to reduce fuel usage
- Water consumption analytics platform
- Sustainability scoring system for businesses
- Environmental policy impact simulator
- Dataset-driven climate trend explorer

Suggested Tech Stack

- Python / Node.js
- Pandas / NumPy / Scikit-learn
- TensorFlow / PyTorch (if needed)
- PostgreSQL / BigQuery / DuckDB
- Mapbox / Leaflet / GIS libraries
- Data visualization libraries
- REST APIs / public datasets
- Offline caching / local storage

Projects to Avoid

- Basic weather apps
- Static awareness websites
- Carbon calculators with fixed formulas
- Simple dashboards without real data
- Fake AI predictions

- Generic sustainability blogs
- Hardcoded datasets only
- Smart home apps without environmental focus
- Projects with no measurable impact
- Apps that require constant internet

CivicTech Track

Description

The CivicTech track focuses on software that improves public systems, governance, accessibility, safety, and community services. Projects should solve real problems faced by citizens, institutions, or public organizations. Systems designed for large populations, limited connectivity, or emergency situations are encouraged. Offline-first, reliable, and scalable solutions are preferred.

Example Project Ideas

- Public grievance prioritization using text analysis
- Disaster response coordination dashboard
- Senior citizen safety monitoring platform
- Government scheme eligibility matcher
- Public transport crowd prediction using data
- Emergency broadcast system for local areas
- Missing person matching using image similarity
- Community volunteer coordination platform
- Road issue clustering from reports
- Queue prediction system for public services

Suggested Tech Stack

- Next.js / React / Flutter (software only)
- Node.js / Python / Java
- PostgreSQL / Supabase / Firebase
- NLP / search / ranking algorithms
- Maps / geolocation APIs
- SMS / notification APIs
- Image / text processing libs
- Offline-first sync / caching

Projects to Avoid

- Complaint form apps
- Static government portals
- FAQ chatbots only

- Basic voting UI without backend
- NGO listing apps
- Pothole reporting clones
- Attendance / form apps
- Fake smart city dashboards
- Projects with no civic use
- Apps that break without internet

Web3 Track

Description

The Web3 track focuses on decentralized software, smart contracts, and trustless systems. Projects should only use blockchain where decentralization, verification, or transparency is required. Strong preference will be given to projects that demonstrate real protocol logic, on-chain design, or distributed architecture rather than token generation.

Example Project Ideas

- On-chain voting with verifiable results
- Decentralized identity verification system
- Smart-contract escrow platform
- DAO governance with proposal execution
- Verifiable certificate storage
- On-chain reputation scoring
- Cross-chain asset tracking dashboard
- Decentralized crowdfunding platform
- Offline-signed transaction system
- Blockchain audit explorer

Suggested Tech Stack

- Solidity / Rust
- Ethereum / Polygon / Solana
- Hardhat / Foundry
- Ethers.js / Web3.js
- IPFS / Filecoin
- WalletConnect / MetaMask
- Next.js / React
- RPC / node providers
- Local testnets / offline signing

Projects to Avoid

- Meme coins
- Token generators
- NFT minting only
- Wallet UI clones
- Copy-paste contracts
- Blockchain without need
- Trading / betting apps
- Fake DAO projects
- AI + Web3 wrappers
- Apps fully dependent on APIs

Recommended Development Tools and Workflow

To help participants focus on building useful and technically strong projects within the limited hackathon duration, the use of modern development tools and AI-assisted workflows is recommended.

UI Development Recommendation

Participants are recommended to start their project by quickly generating a user interface using **v0.dev** or similar UI generation tools. This allows teams to prototype faster and spend more time on core logic, architecture, and real functionality instead of basic UI setup.

AI-Assisted Coding

Participants are encouraged to use AI-assisted development tools such as **Windsurf** or similar coding assistants to improve productivity, code quality, and development speed. These tools may be used for scaffolding, debugging, refactoring, or documentation, but the final project must still demonstrate meaningful logic and engineering effort.

Voice and Audio Integration

Participants are encouraged to integrate voice, speech, or audio-based interaction using tools such as **ElevenLabs** where relevant. Projects that make effective use of voice interfaces, narration, accessibility features, or audio-based interaction will be considered positively, especially in CivicTech and accessibility-related solutions.

Proper Use of AI Tools

AI tools and IDEs may be used during development, but projects must not be simple wrappers around AI APIs. The core value of the project should come from system design, logic, or problem-solving rather than only calling external AI services.

Focus on Core Functionality

Teams are encouraged to:

- Build a working UI early
- Focus on backend logic and real features
- Support offline-first or low-connectivity usage where possible
- Ensure the project works reliably without constant internet dependency
- Avoid spending the entire hackathon on UI styling only

Use of these tools is optional but recommended to help teams build more complete and higher-quality projects within the hackathon duration.